

# Agentic Text-to-SQL System Assignment Report

**Repository link:** *YOUR\_PUBLIC\_GITHUB\_REPO\_LINK\_HERE*

Student: Ina Shrestha

Database: PostgreSQL seeded with the provided Classic Models schema and data.

## What this project does

This project builds the assignment step by step: benchmark SQL answers, query decomposition, Text-to-SQL execution, and a mini agent endpoint. The implementation is rule-based, so it runs without needing an OpenAI/Gemini/Claude API key.

## Task 1 - SQL Benchmark Dataset Preparation

For each benchmark question, I wrote a safe SELECT query, executed it on the seeded database, and saved the result summary. Full results are also saved in reports/ground\_truth\_queries.csv.

| No. | Question                           | Ground Truth SQL                                        | Result Summary                                                                                                                                                                                                                                                    |
|-----|------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | List all products                  | SELECT * FROM products;                                 | 110 rows; sample: [{"productCode": "S10_1678", "productName": "1969 Harley Davidson Ultimate Chopper", "productLine": "Motorcycles", "productScale": "1:10", "productVendor": "Min Lin Diecast", "productDescription": "This replica features working kick..."}]  |
| 2   | Get all customers                  | SELECT * FROM customers;                                | 122 rows; sample: [{"customerNumber": 103, "customerName": "Atelier graphique", "contactLastName": "Schmitt", "contactFirstName": "Carine ", "phone": "40.32.2555", "addressLine1": "54, rue Royale", "addressLine2": null, "city": "Nantes", "state": nul...}]   |
| 3   | Show all orders                    | SELECT * FROM orders;                                   | 326 rows; sample: [{"orderNumber": 10100, "orderDate": "2003-01-06", "requiredDate": "2003-01-13", "shippedDate": "2003-01-10", "status": "Shipped", "comments": null, "customerNumber": 363}, {"orderNumber": 10101, "orderDate": "2003-01-09", "required...}]   |
| 4   | List all employees                 | SELECT * FROM employees;                                | 23 rows; sample: [{"employeeNumber": 1002, "lastName": "Murphy", "firstName": "Diane", "extension": "x5800", "email": "dmurphy@classicmodelcars.com", "officeCode": "1", "reportsTo": null, "jobTitle": "President"}, {"employeeNumber": 1056, "lastName": "...}] |
| 5   | Get all offices                    | SELECT * FROM offices;                                  | 7 rows; sample: [{"officeCode": "1", "city": "San Francisco", "phone": "+1 650 219 4782", "addressLine1": "100 Market Street", "addressLine2": "Suite 300", "state": "CA", "country": "USA", "postalCode": "94080", "territory": "NA"}, {"officeCode": "2"...}]   |
| 6   | Show all product lines             | SELECT * FROM productlines;                             | 7 rows; sample: [{"productLine": "Classic Cars", "textDescription": "Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle cars, dream sports cars or movie-inspired miniatures, you wil...}]   |
| 7   | List all payments                  | SELECT * FROM payments;                                 | 273 rows; sample: [{"customerNumber": 103, "checkNumber": "HQ336336", "paymentDate": "2004-10-19", "amount": 6066.78}, {"customerNumber": 103, "checkNumber": "JM555205", "paymentDate": "2003-06-05", "amount": 14571.44}, {"customerNumber": 103, "check...}]   |
| 8   | Get product names and prices       | SELECT "productName", "buyPrice", "MSRP" FROM products; | 110 rows; sample: [{"productName": "1969 Harley Davidson Ultimate Chopper", "buyPrice": 48.81, "MSRP": 95.7}, {"productName": "1952 Alpine Renault 1300", "buyPrice": 98.58, "MSRP": 214.3}, {"productName": "1996 Moto Guzzi 1100i", "buyPrice": 68.99, "...}]   |
| 9   | Get customer names and cities      | SELECT "customerName", "city" FROM customers;           | 122 rows; sample: [{"customerName": "Atelier graphique", "city": "Nantes"}, {"customerName": "Signal Gift Stores", "city": "Las Vegas"}, {"customerName": "Australian Collectors, Co.", "city": "Melbourne"}]]                                                    |
| 10  | List employee first and last names | SELECT "firstName", "lastName" FROM employees;          | 23 rows; sample: [{"firstName": "Diane", "lastName": "Murphy"}, {"firstName": "Mary", "lastName": "Patterson"}, {"firstName": "Jeff", "lastName": "Firrelli"}]]                                                                                                   |

| No. | Question                                   | Ground Truth SQL                                                                                                                                                                                                                      | Result Summary                                                                                                                                                                                                                                                     |
|-----|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11  | Get all order dates                        | SELECT "orderNumber", "orderDate", "requiredDate", "shippedDate" FROM orders;                                                                                                                                                         | 326 rows; sample: [{"orderNumber": 10100, "orderDate": "2003-01-06", "requiredDate": "2003-01-13", "shippedDate": "2003-01-10"}, {"orderNumber": 10101, "orderDate": "2003-01-09", "requiredDate": "2003-01-18", "shippedDate": "2003-01-11"}, {"orderNumb...      |
| 12  | Show product vendor list                   | SELECT DISTINCT "productVendor" FROM products ORDER BY "productVendor";                                                                                                                                                               | 13 rows; sample: [{"productVendor": "Autoart Studio Design"}, {"productVendor": "Carousel DieCast Legends"}, {"productVendor": "Classic Metal Creations"}]                                                                                                         |
| 13  | Get all product codes                      | SELECT "productCode" FROM products;                                                                                                                                                                                                   | 110 rows; sample: [{"productCode": "S10_1678"}, {"productCode": "S10_1949"}, {"productCode": "S10_2016"}]                                                                                                                                                          |
| 14  | List all countries from offices            | SELECT DISTINCT "country" FROM offices ORDER BY "country";                                                                                                                                                                            | 5 rows; sample: [{"country": "Australia"}, {"country": "France"}, {"country": "Japan"}]                                                                                                                                                                            |
| 15  | Show all order statuses                    | SELECT DISTINCT "status" FROM orders ORDER BY "status";                                                                                                                                                                               | 6 rows; sample: [{"status": "Cancelled"}, {"status": "Disputed"}, {"status": "In Process"}]                                                                                                                                                                        |
| 16  | Get all payment amounts                    | SELECT "amount" FROM payments;                                                                                                                                                                                                        | 273 rows; sample: [{"amount": 6066.78}, {"amount": 14571.44}, {"amount": 1676.14}]                                                                                                                                                                                 |
| 17  | List all job titles                        | SELECT DISTINCT "jobTitle" FROM employees ORDER BY "jobTitle";                                                                                                                                                                        | 7 rows; sample: [{"jobTitle": "President"}, {"jobTitle": "Sale Manager (EMEA)"}, {"jobTitle": "Sales Manager (APAC)"}]                                                                                                                                             |
| 18  | Get customer phone numbers                 | SELECT "customerName", "phone" FROM customers;                                                                                                                                                                                        | 122 rows; sample: [{"customerName": "Atelier graphique", "phone": "40.32.2555"}, {"customerName": "Signal Gift Stores", "phone": "7025551838"}, {"customerName": "Australian Collectors, Co.", "phone": "03 9520 4555"}]                                           |
| 19  | Show product MSRP values                   | SELECT "productName", "MSRP" FROM products;                                                                                                                                                                                           | 110 rows; sample: [{"productName": "1969 Harley Davidson Ultimate Chopper", "MSRP": 95.7}, {"productName": "1952 Alpine Renault 1300", "MSRP": 214.3}, {"productName": "1996 Moto Guzzi 1100i", "MSRP": 118.94}]                                                   |
| 20  | List order numbers                         | SELECT "orderNumber" FROM orders;                                                                                                                                                                                                     | 326 rows; sample: [{"orderNumber": 10100}, {"orderNumber": 10101}, {"orderNumber": 10102}]                                                                                                                                                                         |
| 21  | Get orders with customer names             | SELECT o."orderNumber", o."orderDate", o."status", c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";                                                                                        | 326 rows; sample: [{"orderNumber": 10100, "orderDate": "2003-01-06", "status": "Shipped", "customerName": "Online Diecast Creations Co."}, {"orderNumber": 10101, "orderDate": "2003-01-09", "status": "Shipped", "customerName": "Blauer See Auto, Co."}, ...]    |
| 22  | Get employees with office city             | SELECT e."firstName", e."lastName", o."city" FROM employees e JOIN offices o ON e."officeCode" = o."officeCode";                                                                                                                      | 23 rows; sample: [{"firstName": "Diane", "lastName": "Murphy", "city": "San Francisco"}, {"firstName": "Mary", "lastName": "Patterson", "city": "San Francisco"}, {"firstName": "Jeff", "lastName": "Firrelli", "city": "San Francisco"}]                          |
| 23  | Get payments with customer names           | SELECT p."checkNumber", p."paymentDate", p."amount", c."customerName" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";                                                                                    | 273 rows; sample: [{"checkNumber": "HQ336336", "paymentDate": "2004-10-19", "amount": 6066.78, "customerName": "Atelier graphique"}, {"checkNumber": "JM555205", "paymentDate": "2003-06-05", "amount": 14571.44, "customerName": "Atelier graphique"}, {"...}]    |
| 24  | Get order details with product names       | SELECT od."orderNumber", p."productName", od."quantityOrdered", od."priceEach" FROM orderdetails od JOIN products p ON od."productCode" = p."productCode";                                                                            | 2996 rows; sample: [{"orderNumber": 10100, "productName": "1917 Grand Touring Sedan", "quantityOrdered": 30, "priceEach": 136}, {"orderNumber": 10100, "productName": "1911 Ford Town Car", "quantityOrdered": 50, "priceEach": 55.09}, {"orderNumber": 10...}]    |
| 25  | Get products with product line description | SELECT p."productName", p."productLine", pl."textDescription" FROM products p JOIN productlines pl ON p."productLine" = pl."productLine";                                                                                             | 110 rows; sample: [{"productName": "1969 Harley Davidson Ultimate Chopper", "productLine": "Motorcycles", "textDescription": "Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson,...}]    |
| 26  | Get customers with sales rep names         | SELECT c."customerName", e."firstName", e."lastName" FROM customers c LEFT JOIN employees e ON c."salesRepEmployeeNumber" = e."employeeNumber";                                                                                       | 122 rows; sample: [{"customerName": "Atelier graphique", "firstName": "Gerard", "lastName": "Hernandez"}, {"customerName": "Signal Gift Stores", "firstName": "Leslie", "lastName": "Thompson"}, {"customerName": "Australian Collectors, Co.", "firstName": ...}] |
| 27  | Get orders with customer city              | SELECT o."orderNumber", o."orderDate", o."status", c."city" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";                                                                                                | 326 rows; sample: [{"orderNumber": 10100, "orderDate": "2003-01-06", "status": "Shipped", "city": "Nashua"}, {"orderNumber": 10101, "orderDate": "2003-01-09", "status": "Shipped", "city": "Frankfurt"}, {"orderNumber": 10102, "orderDate": "2003-01-10"...}]    |
| 28  | Get employees and their manager            | SELECT e."firstName" AS employee_first_name, e."lastName" AS employee_last_name, m."firstName" AS manager_first_name, m."lastName" AS manager_last_name FROM employees e LEFT JOIN employees m ON e."reportsTo" = m."employeeNumber"; | 23 rows; sample: [{"employee_first_name": "Diane", "employee_last_name": "Murphy", "manager_first_name": null, "manager_last_name": null}, {"employee_first_name": "Mary", "employee_last_name": "Patterson", "manager_first_name": "Diane", "manager_last...}]    |
| 29  | Get orderdetails with product vendor       | SELECT od."orderNumber", od."productCode", p."productVendor", od."quantityOrdered" FROM orderdetails od JOIN products p ON od."productCode" = p."productCode";                                                                        | 2996 rows; sample: [{"orderNumber": 10100, "productCode": "S18_1749", "productVendor": "Welly Diecast Productions", "quantityOrdered": 30}, {"orderNumber": 10100, "productCode": "S18_2248", "productVendor": "Motor City Art Classics", "quantityOrdered": ...}] |
| 30  | Get payments with customer country         | SELECT p."checkNumber", p."paymentDate", p."amount", c."country" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";                                                                                         | 273 rows; sample: [{"checkNumber": "HQ336336", "paymentDate": "2004-10-19", "amount": 6066.78, "country": "France"}, {"checkNumber": "JM555205", "paymentDate": "2003-06-05", "amount": 14571.44, "country": "France"}, {"checkNumber": "OM314933", "payme...}]    |
| 31  | Count customers per country                | SELECT "country", COUNT("customerNumber") AS customer_count FROM customers GROUP BY "country" ORDER BY customer_count DESC, "country";                                                                                                | 28 rows; sample: [{"country": "USA", "customer_count": 36}, {"country": "Germany", "customer_count": 13}, {"country": "France", "customer_count": 12}]                                                                                                             |
| 32  | Total payments per customer                | SELECT c."customerName", SUM(p."amount") AS total_payments FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber" GROUP BY c."customerName" ORDER BY total_payments DESC;                                        | 98 rows; sample: [{"customerName": "Euro+ Shopping Channel", "total_payments": 715738.98}, {"customerName": "Mini Gifts Distributors Ltd.", "total_payments": 584188.24}, {"customerName": "Australian Collectors, Co.", "total_payments": 180585.07}]             |
| 33  | Number of orders per status                | SELECT "status", COUNT("orderNumber") AS order_count FROM orders GROUP BY "status" ORDER BY order_count DESC, "status";                                                                                                               | 6 rows; sample: [{"status": "Shipped", "order_count": 303}, {"status": "Cancelled", "order_count": 6}, {"status": "In Process", "order_count": 6}]                                                                                                                 |

| No. | Question                           | Ground Truth SQL                                                                                                                                                                                                                     | Result Summary                                                                                                                                                                                                                   |
|-----|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 34  | Products per product line          | SELECT "productLine", COUNT("productCode") AS product_count FROM products GROUP BY "productLine" ORDER BY product_count DESC, "productLine";                                                                                         | 7 rows; sample: [{"productLine": "Classic Cars", "product_count": 38}, {"productLine": "Vintage Cars", "product_count": 24}, {"productLine": "Motorcycles", "product_count": 13}]                                                |
| 35  | Employees per office               | SELECT o."officeCode", o."city", COUNT(e."employeeNumber") AS employee_count FROM offices o LEFT JOIN employees e ON o."officeCode" = e."officeCode" GROUP BY o."officeCode", o."city" ORDER BY employee_count DESC, o."officeCode"; | 7 rows; sample: [{"officeCode": "1", "city": "San Francisco", "employee_count": 6}, {"officeCode": "4", "city": "Paris", "employee_count": 5}, {"officeCode": "6", "city": "Sydney", "employee_count": 4}]                       |
| 36  | Total stock per product vendor     | SELECT "productVendor", SUM("quantityInStock") AS total_stock FROM products GROUP BY "productVendor" ORDER BY total_stock DESC, "productVendor";                                                                                     | 13 rows; sample: [{"productVendor": "Gearbox Collectibles", "total_stock": 60495}, {"productVendor": "Min Lin Diecast", "total_stock": 50089}, {"productVendor": "Classic Metal Creations", "total_stock": 45408}]               |
| 37  | Average buy price per product line | SELECT "productLine", ROUND(AVG("buyPrice"), 2) AS average_buy_price FROM products GROUP BY "productLine" ORDER BY "productLine";                                                                                                    | 7 rows; sample: [{"productLine": "Classic Cars", "average_buy_price": 64.45}, {"productLine": "Motorcycles", "average_buy_price": 50.69}, {"productLine": "Planes", "average_buy_price": 49.63}]                                 |
| 38  | Orders per customer                | SELECT c."customerName", COUNT(o."orderNumber") AS order_count FROM customers c LEFT JOIN orders o ON c."customerNumber" = o."customerNumber" GROUP BY c."customerName" ORDER BY order_count DESC, c."customerName";                 | 122 rows; sample: [{"customerName": "Euro+ Shopping Channel", "order_count": 26}, {"customerName": "Mini Gifts Distributors Ltd.", "order_count": 17}, {"customerName": "Australian Collectors, Co.", "order_count": 5}]         |
| 39  | Max MSRP per product line          | SELECT "productLine", MAX("MSRP") AS max_msrp FROM products GROUP BY "productLine" ORDER BY max_msrp DESC, "productLine";                                                                                                            | 7 rows; sample: [{"productLine": "Classic Cars", "max_msrp": 214.3}, {"productLine": "Motorcycles", "max_msrp": 193.66}, {"productLine": "Vintage Cars", "max_msrp": 170}]                                                       |
| 40  | Min buy price per vendor           | SELECT "productVendor", MIN("buyPrice") AS min_buy_price FROM products GROUP BY "productVendor" ORDER BY min_buy_price ASC, "productVendor";                                                                                         | 13 rows; sample: [{"productVendor": "Carousel DieCast Legends", "min_buy_price": 15.91}, {"productVendor": "Second Gear Diecast", "min_buy_price": 16.24}, {"productVendor": "Classic Metal Creations", "min_buy_price": 20.61}] |
| 41  | Total number of customers          | SELECT COUNT("customerNumber") AS total_customers FROM customers;                                                                                                                                                                    | 122                                                                                                                                                                                                                              |
| 42  | Total number of products           | SELECT COUNT("productCode") AS total_products FROM products;                                                                                                                                                                         | 110                                                                                                                                                                                                                              |
| 43  | Total revenue from payments        | SELECT SUM("amount") AS total_revenue FROM payments;                                                                                                                                                                                 | 8853839.23                                                                                                                                                                                                                       |
| 44  | Average product price              | SELECT ROUND(AVG("buyPrice"), 2) AS average_product_buy_price FROM products;                                                                                                                                                         | 54.4                                                                                                                                                                                                                             |
| 45  | Max payment amount                 | SELECT MAX("amount") AS max_payment_amount FROM payments;                                                                                                                                                                            | 120166.58                                                                                                                                                                                                                        |
| 46  | Min payment amount                 | SELECT MIN("amount") AS min_payment_amount FROM payments;                                                                                                                                                                            | 615.45                                                                                                                                                                                                                           |
| 47  | Count total orders                 | SELECT COUNT("orderNumber") AS total_orders FROM orders;                                                                                                                                                                             | 326                                                                                                                                                                                                                              |
| 48  | Total quantity in stock            | SELECT SUM("quantityInStock") AS total_quantity_in_stock FROM products;                                                                                                                                                              | 555131                                                                                                                                                                                                                           |
| 49  | Average MSRP                       | SELECT ROUND(AVG("MSRP"), 2) AS average_msrp FROM products;                                                                                                                                                                          | 100.44                                                                                                                                                                                                                           |
| 50  | Number of employees                | SELECT COUNT("employeeNumber") AS number_of_employees FROM employees;                                                                                                                                                                | 23                                                                                                                                                                                                                               |

## Task 2 - Query Understanding / Decomposition

Each question was broken into structured components: intent, tables, columns, filters, and joins. The full decomposition file is reports/task2\_decompositions.csv. Example:

|          |                                                  |
|----------|--------------------------------------------------|
| Question | Get orders with customer names                   |
| Intent   | Retrieve orders with customer names              |
| Tables   | orders, customers                                |
| Columns  | orderNumber, orderDate, status, customerName     |
| Filters  | None                                             |
| Joins    | orders.customerNumber = customers.customerNumber |

## Task 3 - Text-to-SQL Pipeline

The pipeline follows this sequence: natural language question -> structured decomposition -> SQL generation -> safe validation -> PostgreSQL execution -> retry once if needed -> structured JSON output.

Important design choices:

- The system uses a benchmark rule bank instead of an external LLM so it works for every student without API keys.
- All generated SQL must start with SELECT. INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, and other unsafe keywords are blocked.
- Execution is logged in logs/agent.log with decomposition, SQL generation, execution status, and execution time.
- Task 3 retry limit is one retry, matching the assignment requirement.

## Task 4 - Mini SQL Agent

The mini agent is exposed at POST /agent/sql. It follows: understand query, generate SQL, execute SQL, handle errors, retry up to 3 times, and return a natural-language summary.

Example request:

```
{ "question": "Total number of customers" }
```

Example output style:

```
{ "sql": "SELECT COUNT(\"customerNumber\") AS total_customers FROM customers;", "result": [{"total_customers": 122}], "summary": "The answer to Total number of customers is 122.", "status": "success" }
```

## Evaluation Strategy

| Metric                 | Meaning                                                                     |
|------------------------|-----------------------------------------------------------------------------|
| Execution success rate | Percentage of benchmark questions that run without database errors.         |
| Result correctness     | Whether the returned result matches the manually verified ground-truth SQL. |
| Table/column selection | Whether the generated SQL uses the correct schema objects.                  |
| Join correctness       | Whether join conditions match foreign key relationships.                    |

|                                 |                                                  |
|---------------------------------|--------------------------------------------------|
| Retry success rate              | Whether failed SQL gets corrected successfully.  |
| Latency                         | Time taken to generate and execute each query.   |
| Safety                          | Whether non-SELECT or harmful SQL is blocked.    |
| Natural-language answer quality | Whether the summary explains the result clearly. |

## How to Run

1. Open Docker Desktop. 2. Open this project folder in VS Code. 3. Run: `docker compose up --build`. 4. Open <http://localhost:8000/docs>. 5. Test `/decompose`, `/text-to-sql`, and `/agent/sql`. 6. Run evaluation with: `docker compose exec api python scripts/evaluate.py`. 7. Upload this project to a public GitHub repo and paste the link on the first page of this PDF.